Marc Olson

Econ ML Prediction Project

Data expansion

Before diving too deeply into the data, I first transformed final bid to its natural log, and omitted rowid, link url, and pulled at from prediction (pulled at was used to determine CV folds). I next investigated data from outside sources, and had Claude scrape the number of pieces a given artist has in the Art Institute of Chicago and The Met as a proxy for artist notoriety. I also was interested in the dimensions of each piece as listed on its auction page, but the site is pretty strictly against scraping and bot-based tools, so this was a dead end. Considering the work I did with the preexisting data, I am not confident that other outside sources would add much more predictive power, so I stopped here in terms of outside expansions.

My primary data objective was turning the text variables (link text, condition, series text) into something with more predictive power. It seemed obvious that the best method would not be cherry-picking words and phrases that occur to me as important, but letting the model identify which parts hold the most predictive power. This forced me to gain a baseline familiarity with text tokenization and apply it to the link text, condition, and series text columns. Before this process, I extracted the four clearance sentences and mention of $25 sales from entries, and turned each into indicators (clearance1 - 4, sale25). Statements identifying that an item is on sale likely have more predictive power than other text, and since there are only a few unique sale statements, they were easy to turn into binary variables.

Generally, tokenization consisted of breaking every full text cell into every single word and two-word combination. Before this, I first eliminated articles from the text fields, as I would anticipate their predictive power to be very low. I considered using a pre-existing list of "stop words," but after looking into scikit-learn's stop word list, I felt it was eliminating too many words, and decided to stick with articles. For the series text specifically, I also filled in NAs with a space so these values didn't get dropped, and included a "series present" indicator variable to differentiate between uninformative series text entries and entries that were blank from the start, as discussed later. After tokenization, I applied TF-IDF weighting to these tokens to weight each token by the rate at which it appears in an entry relative to how long the entry is and the rate at which it appears across all the entries in the data.

This tokenization process creates a huge number of variables which might clutter the model, with most variables likely bringing little to nothing to the table. To first trim some of the extremely rare ones, I began by eliminating tokens that appeared less than five times in the training data (this is what motivated the empty series indicator in series text) to prevent tokens associated with a single or very small number of prints from being treated as useful for general prediction. From here, I used two strategies to streamline the token clutter, and turned them both into cross-validation

options. First, I used only the tokens from a given column in a lasso model to predict out-of-fold log final bid, turning the tokens into a single value to feed the model. For the second strategy, I instead turned the number of tokens retained into a tuning parameter, meaning that the cross-validation procedure will identify the number of text tokens that minimize the sum of squared residuals alongside other tuning parameters, retaining different N tokens ranked by their predictive power in the lasso model. Both of these strategies were compared in the CV procedure discussed below.

This process also mapped pretty well onto the artist info column, not only because tokenization could help capture the predictive power of a single artist or artist collaborations, but it would also keep artists that only appear a few times in the training data from generating extreme predictions. Before I applied this process, I first extracted any year present in the artist field. Then, I applied the tokenization process to get the single number score only, as I would not expect adding more N predictors to be very helpful for artists, there are far fewer artists than there are unique tokens in the link text field. Importantly, I did not use TF-IDF weighting here, but I did create a N entries variable that gives the total number of pieces that artist has in the training set, as frequency of appearance is likely important. This procedure also accommodates the existence of artists in the testing data that are not in the training data, as I am not including artist names directly, but the predictive power of their names. Artists in the test data that are not present in the training data would then be treated like a training artist that doesn't contribute much to prediction.

The final data hurdle was the year a piece was created, the year ranges of the artists, and the numerous ways that was captured in the data. From print to print, there could be a specific year, a year range, a decade, or any of those options with a "circa". To consolidate this into a single year for a print and a range for an artist, I first extracted the years from link text, artist info, and series text. From there, I created a made year column which considered the year from link text or series text, prioritizing years from link text then series, and taking the midpoint of a range if a range was given. For artist year, I created one artist year column based on their year values, taking the midpoint if a range was given and the single value if a single year was given. For all year columns, I also included a circa indicator if circa was used to describe a year, and an active indicator to differentiate between "artist active" and "artist alive" ranges for artists. Finally, only for artists that did not have an "active" flag and had a two-number lifespan range, I made an indicator for prints that were listed in a year after the associated artist's death. My thought here is that prints/artists with more ambiguity surrounding their work/lifespan, or prints that were made close to the end of an artist's life might be rarer or hold more value. Finally, I filled in blanks in these columns with -1 for random forests, and the median year for lasso, ridge, and OLS models.

See the end of the report for a full table of the rows in the final dataset.

CV and model selection

With the data prepped, I turned to cross-validation, model selection, and tuning. I initially had a random forest in mind as the best-fitting model for this scenario due to the text-based variables and the likely presence of complex interactions. But, I also decided to include ridge, lasso, and OLS models as comparisons, as similar predictive accuracy would suggest that relationships in the data are more linear, and an RF isn't necessary. The lasso model is of particular interest because I turned the text columns into a huge number of variables, and it is likely that many are not valuable for prediction. The lasso model would address this well, and could shrink these unimportant predictors towards zero. For lasso and ridge models specifically, the only parameter in need of tuning is lambda, and the selected model is the one corresponding to the lambda value within one standard error of the minimum to prevent overfitting. For the random forest, the tuning parameters are mtry, the number of variables randomly selected as candidates for each split point, and minimum node size, the minimum number of observations in a given node that can still be split.

For the RF models, I tuned the N terms from the tokenization processes described above, tuning across 0, 25, 50, and 100 terms retained. This doesn't apply to the lasso and ridge models, because they do that selection themselves. Importantly, these terms are tuned simultaneously, meaning that the link text terms, condition, and series info are all 25, all 50, and all 100, instead of all possible combinations of the three values, to reduce the time required for tuning. Also, the lasso-based scores computed based on each token are included in tuning, through included/not included conditions. For the random forest models, I also tuned the number of features (5, 10, 15, 20, 30, 40, 50, 60) and minimum node size (3, 5, 7, 10). All RFs were computed with 500 trees.

The tuning procedure was conducted using five-fold cross-validation while respecting the temporal ordering of the data to prevent leakage. Specifically, this meant the training data folds were time-ordered, and past folds were used to predict the one fold ahead of it in time. Further, since the text tokenization variables are based on all the rows in the data together, I rebuilt the token variables within each CV fold to prevent leakage. The CV process aimed to maximize the SSR reduction compared to predicting the mean natural log final bid.

Across all models, the best SSR reductions were achieved when N tokens was 100. The RF models had the greatest SSR reduction in the CV process overall, and the best RF model reduced the SSR by 0.762, with 50 features and a minimum node size of 3. Other RF models close to this SSR reduction were nearly identical, with the same N tokens, omitting the lasso token scores, but differing in number of features and minimum node size. But, since SSR reduction didn't move much between the top RF models, I would conclude that feature count is relatively unimportant for this model's predictive ability. The ridge, lasso, and OLS models were not close to the RF, which suggests that the relationships in this data contain complex interactions and are nonlinear. The below tables show the top model in terms of SSR removed for the lasso, ridge, and OLS models, and the top 5 models for RF.

Model	Lasso scores	N tokens	Features	Min.Node Size	SSR Reduction
RF	Excluded	100	50	3	0.762
RF	Excluded	100	40	3	0.762
RF	Excluded	100	30	3	0.761
RF	**Excluded**	**100**	**50**	**5**	**0.761**
RF	Excluded	100	15	3	0.761
Lasso	Included	100	-	-	0.666
Ridge	Included	100	-	-	0.662
OLS	Included	100	-	-	0.662

RF Diagnostics

Based on the CV outputs, I selected the simplest model within what I considered to be noise in SSR reduction, which was the 4th best model in Table 1 (bolded). Although random forest models can be difficult to interpret plainly, and this is made slightly more difficult without access to the actual test values, there are still a few methods available to interpret the final model. Specifically, variable importance plots and partial dependence plots can be used to identify which variables are more important for generating predictions relative to others, and how the predicted value of the outcome changes as a given variable value changes. Since there are hundreds of predictors in the final dataset, I cut the VIP to the top 20 most important predictors and created PDPs for the top four most important predictors.

The VIP shows that the artist lasso score is far and away the most important predictor, followed by one of the clearance statements, artist year, and made year. Also, the top 20 contained 8 condition-related text tokens. Taken together, this suggests that the artist, the year the print was made, its condition, and whether it was on clearance are doing the most predictive work. The PDPs for these top four variables add a bit more context, identifying that predicted natural log of final bid increases in a linear fashion as artist score increases, and is far higher when clearance statement 1 is not present. Further, the two year variables suggest that the more important piece of the variable from a predictive standpoint is the presence of an artist year or made year, with -1 values showing different predictions than those with an actual year, where predictions are flatter. It may be that prints without a year are of less interest, and sellers didn't care to identify the specific year of an artist/print because they already know it isn't going to sell for as much. It makes sense in an environment like this for more expensive prints to have more information given about them, to help bidders justify a high bid, and that is what I think I am picking up here.

Overall, these results are fairly intuitive, and don't offer much insight past that which could be guessed quite easily: certain artists are generally going to have more expensive prints, condition has a strong connection to price, prints on clearance are likely to be less expensive. However, I think the RF model does a good job of considering these conditions at the same time, and capturing interactions between them.

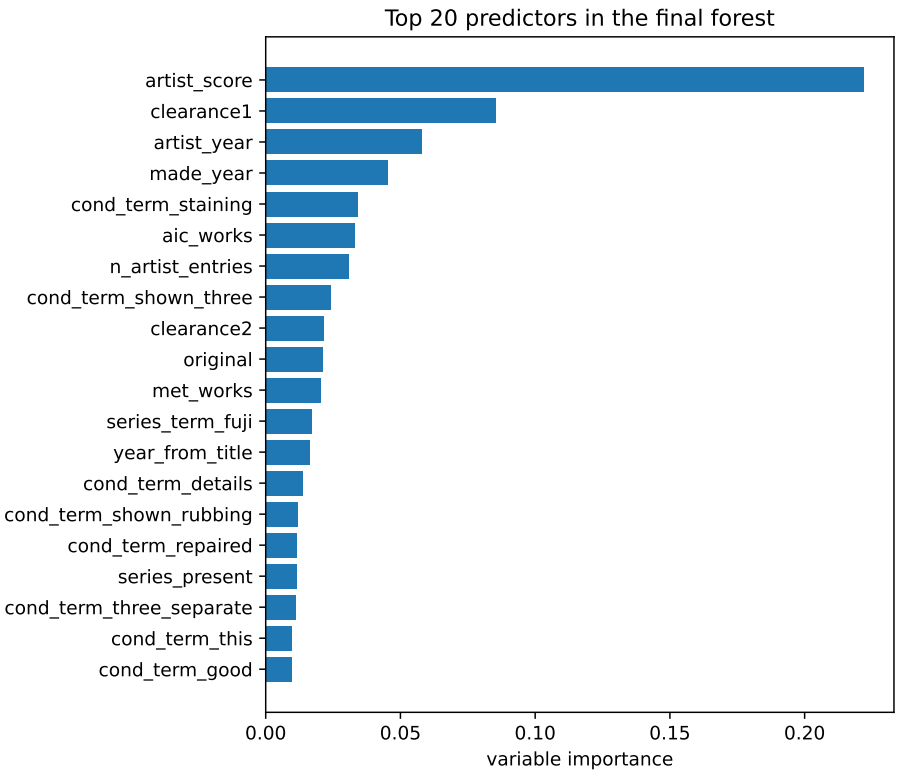


Figure 1: VIP of top 20 predictors

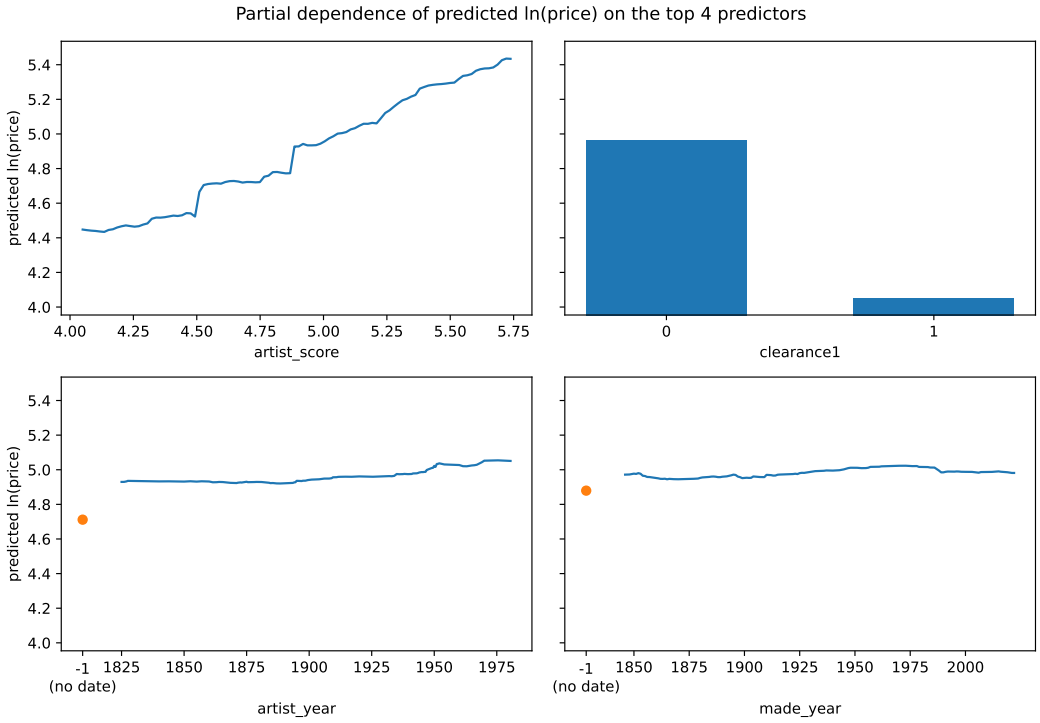


Figure 2: PDP of top 4 predictors

Variable	Type	Description
artist_score	number	Lasso prediction of outcome from the tokenization process
original	0/1	Original print indicator
made_year	number	Print year, missing = -1
year_from_title	0/1	made_year came from title
year_from_series	0/1	made_year came from series
year_from_artist	0/1	made_year came from artist
posthumous	0/1	Print dated after the artist's death
artist_year	number	Midpoint of the artist's life/active dates, missing = -1
artist_circa	0/1	Artist dates described with circa
artist_active	0/1	Artist dates described with active
n_artist_entries	count	N prints by a given artist
aic_works	count	Works by the artist in the Art Institute of Chicago, missing artists = 0 (collected 11/4/26; api.artic.edu)
met_works	count	Works by the artist in the Metropolitan Museum of Art, missing artists = 0 (collected 11/4/26; github.com/metmuseum/openaccess)
title_circa	0/1	Title date marked circa
series_circa	0/1	Series date marked circa
series_present	0/1	Print has a non-NA series_text
sale_25	0/1	Series text mentions $25
clearance1–4	0/1	One indicator per clearance title statement
cond_term_* (100)	0/1	Condition text contains a given token
title_term_* (100)	0/1	Title contains a given token
series_term_* (100)	0/1	Series contains a given token